



MATLAB ANALYSIS OF A 1 kW MOSFET POWER AMPLIFIER

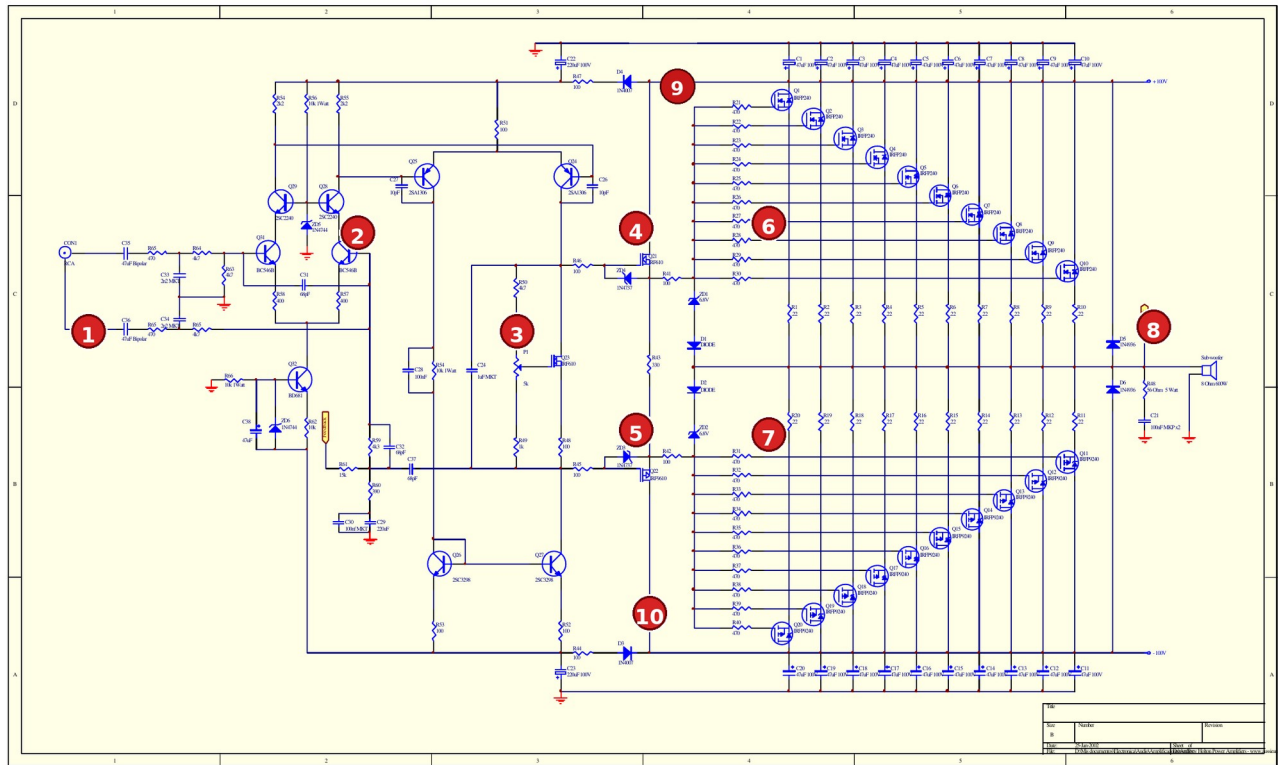
Frequency Sweep • Test Points • Gain • Phase • Output Power

1. Objective / 1. Objectif

Measure the amplifier at defined test points while applying a controlled sine-wave frequency sweep. MATLAB is then used to calculate and visualize voltage gain, phase, output power, internal-stage response and supply-rail behavior.

Mesurer l'amplificateur à des points de test définis pendant un balayage fréquentiel sinusoïdal contrôlé. MATLAB sert ensuite à calculer et visualiser le gain en tension, la phase, la puissance de sortie, la réponse des étages internes et le comportement des rails d'alimentation.

2. Amplifier Under Test / Amplificateur à l'essai



Original 2002 schematic with MATLAB measurement points TP1-TP10 / Schéma original de 2002 avec points de mesure MATLAB TP1-TP10

Important: the numbered measurement convention is now fixed as TP1 input, TP2-TP7 internal/driver points, TP8 output, TP9 positive rail and TP10 negative rail. TP-GND and TP-I remain auxiliary references and are not part of the TP1-TP10 numbering. / Important : la convention numérotée est maintenant fixée ainsi : TP1 entrée, TP2-TP7 points internes/drivers, TP8 sortie, TP9 rail positif et TP10 rail négatif. TP-GND et TP-I restent des références auxiliaires et ne font pas partie de la numérotation TP1-TP10.

3. Measurement Points / Points de mesure

Point	Location / Emplacement	Purpose / Utilité
TP1	Input / Entrée	Input sine-wave reference / Référence sinusoïdale d'entrée
TP2	Internal stage 1 / Étage interne 1	Internal RMS voltage / Tension RMS



		interne
TP3	Internal stage 2 / Étage interne 2	Internal RMS voltage / Tension RMS interne
TP4	Upper driver path / Driver supérieur	Upper driver RMS voltage / Tension RMS driver supérieur
TP5	Lower driver path / Driver inférieur	Lower driver RMS voltage / Tension RMS driver inférieur
TP6	Upper gate-drive bus / Commande grille supérieure	Upper gate-drive RMS voltage / Tension RMS commande supérieure
TP7	Lower gate-drive bus / Commande grille inférieure	Lower gate-drive RMS voltage / Tension RMS commande inférieure
TP8	Amplifier output / Sortie amplificateur	Output RMS voltage Vout / Tension RMS de sortie Vout
TP9	Positive supply rail / Rail positif	Positive rail voltage +V / Tension du rail positif +V
TP10	Negative supply rail / Rail négatif	Negative rail voltage -V / Tension du rail négatif -V
Aux.	TP-GND / TP-I	Common reference and optional load-current measurement / Référence commune et mesure optionnelle du courant de charge

4. Test Conditions / 4. Conditions d'essai

Use a sine wave with constant input amplitude. Suggested sweep: 100, 200, 500 Hz, 1, 2, 5, 10, 15 and 20 kHz. Record RMS values only after the waveform is stable and remains below clipping. Start at low level and increase power only within the safe limits of the amplifier, load and test equipment.	Utiliser une sinusoïde d'amplitude d'entrée constante. Balayage suggéré : 100, 200, 500 Hz, 1, 2, 5, 10, 15 et 20 kHz. Relever les valeurs RMS seulement lorsque la forme d'onde est stable et demeure sous l'écrtage. Commencer à faible niveau et augmenter la puissance uniquement dans les limites sûres de l'amplificateur, de la charge et des instruments.
--	---

5. MATLAB Calculations / 5. Calculs MATLAB

The supplied script supports manual entry and CSV import. It calculates linear voltage gain, gain in dB, expected load current for an 8 Ω resistive load, output power, and produces frequency-response plots. Optional phase, internal TP and supply-rail data are plotted only when measurements are available.	Le script fourni accepte la saisie manuelle et l'importation CSV. Il calcule le gain linéaire, le gain en dB, le courant de charge attendu pour une charge résistive de 8 Ω , la puissance de sortie et génère les courbes de réponse fréquentielle. Les données optionnelles de phase, des TP internes et des rails sont tracées seulement lorsqu'elles sont disponibles.
--	---

6. MATLAB Commands Used / Commandes MATLAB utilisées

MATLAB command	Meaning / Signification
clear	Removes workspace variables / Supprime les variables de l'espace de travail
close all	Closes figure windows / Ferme les fenêtres de graphiques
clc	Clears Command Window / Efface la fenêtre de commande
readtable	Imports CSV data / Importe les données CSV
table	Creates a results table / Crée une table de résultats
semilogx	Plots with logarithmic X axis / Trace avec axe X logarithmique
xlabel / ylabel	Labels graph axes / Nomme les axes
title	Adds graph title / Ajoute un titre
grid on	Displays graph grid / Affiche la grille
legend	Identifies plotted traces / Identifie les courbes
writetable	Exports results to CSV / Exporte les résultats en CSV

7. MATLAB Program / Programme MATLAB

Complete MATLAB source code is reproduced below exactly as the working .m file, with English line-by-line comments. The program supports manual data entry or CSV import and uses the 100 Hz to 20 kHz sine-wave sweep defined in this document. / Le code source MATLAB complet est reproduit ci-dessous tel qu'il apparaît dans le fichier .m, avec des commentaires explicatifs en anglais. Le programme accepte la saisie manuelle ou l'importation CSV et utilise le balayage sinusoïdal de 100 Hz à 20 kHz défini dans ce document.



```
%% 1 kW MOSFET Power Amplifier - Measurement Analysis
% AHDS Engineering Reference
% This script analyzes measured amplifier data versus frequency.
% Replace the example measurement arrays with your own measured values.
% The script can also import the same data from a CSV file.
%
% Expected CSV column names:
% Frequency_Hz, Vin_RMS_V, TP2_RMS_V, TP3_RMS_V, TP4_RMS_V,
% TP5_RMS_V, TP6_RMS_V, TP7_RMS_V, Vout_RMS_V, Iout_RMS_A,
% Vrail_Pos_V, Vrail_Neg_V, Phase_deg
%
% IMPORTANT:
% The internal TP2-TP7 labels are placeholders until the final annotated
% original schematic is inserted and each point is electrically verified.

%% User settings

% Clear variables from the MATLAB workspace.
clear;

% Close all open figure windows.
close all;

% Clear the MATLAB Command Window.
clc;

% Select how the measurement data will be loaded.
% Use "manual" to enter values directly below.
% Use "csv" to import values from a CSV file.
dataMode = "manual";

% Define the CSV file name used when dataMode is set to "csv".
csvFileName = "amplifier_measurements.csv";

% Define the nominal resistive load in ohms.
loadResistance_Ohm = 8;

%% Load measurement data

% Check whether manual data entry was selected.
if dataMode == "manual"

    % Define the test frequencies in hertz.
    Frequency_Hz = [100 200 500 1000 2000 5000 10000 15000 20000];

    % Enter the measured RMS input voltage at every test frequency.
    Vin_RMS_V = [1 1 1 1 1 1 1 1 1];

    % Enter the measured RMS voltage at internal test point TP2.
    TP2_RMS_V = nan(size(Frequency_Hz));

    % Enter the measured RMS voltage at internal test point TP3.
    TP3_RMS_V = nan(size(Frequency_Hz));

    % Enter the measured RMS voltage at internal test point TP4.
    TP4_RMS_V = nan(size(Frequency_Hz));

    % Enter the measured RMS voltage at internal test point TP5.
    TP5_RMS_V = nan(size(Frequency_Hz));

    % Enter the measured RMS voltage at upper gate-drive test point TP6.
    TP6_RMS_V = nan(size(Frequency_Hz));

    % Enter the measured RMS voltage at lower gate-drive test point TP7.
    TP7_RMS_V = nan(size(Frequency_Hz));

    % Enter the measured RMS amplifier output voltage.
    Vout_RMS_V = nan(size(Frequency_Hz));
```



```
% Enter the measured RMS load current.
% Leave values as NaN if current was not measured directly.
Iout_RMS_A = nan(size(Frequency_Hz));

% Enter the measured positive supply-rail voltage.
Vrail_Pos_V = nan(size(Frequency_Hz));

% Enter the measured negative supply-rail voltage as a negative value.
Vrail_Neg_V = nan(size(Frequency_Hz));

% Enter the measured output phase relative to the input in degrees.
Phase_deg = nan(size(Frequency_Hz));

% Check whether CSV import was selected.
elseif dataMode == "csv"

    % Read the measurement table from the selected CSV file.
    T = readtable(csvFileName);

    % Copy the frequency column from the imported table.
    Frequency_Hz = T.Frequency_Hz;

    % Copy the RMS input-voltage column from the imported table.
    Vin_RMS_V = T.Vin_RMS_V;

    % Copy the internal TP2 voltage column from the imported table.
    TP2_RMS_V = T.TP2_RMS_V;

    % Copy the internal TP3 voltage column from the imported table.
    TP3_RMS_V = T.TP3_RMS_V;

    % Copy the internal TP4 voltage column from the imported table.
    TP4_RMS_V = T.TP4_RMS_V;

    % Copy the internal TP5 voltage column from the imported table.
    TP5_RMS_V = T.TP5_RMS_V;

    % Copy the upper gate-drive TP6 voltage column from the imported table.
    TP6_RMS_V = T.TP6_RMS_V;

    % Copy the lower gate-drive TP7 voltage column from the imported table.
    TP7_RMS_V = T.TP7_RMS_V;

    % Copy the RMS output-voltage column from the imported table.
    Vout_RMS_V = T.Vout_RMS_V;

    % Copy the RMS output-current column from the imported table.
    Iout_RMS_A = T.Iout_RMS_A;

    % Copy the positive supply-rail voltage column from the imported table.
    Vrail_Pos_V = T.Vrail_Pos_V;

    % Copy the negative supply-rail voltage column from the imported table.
    Vrail_Neg_V = T.Vrail_Neg_V;

    % Copy the phase column from the imported table.
    Phase_deg = T.Phase_deg;

% Stop the script if an unsupported data mode was entered.
else
    error('dataMode must be "manual" or "csv".');
end

%% Validate the measurement arrays

% Store the number of frequency points for later consistency checks.
numberOfPoints = numel(Frequency_Hz);
```



```
% Verify that the input-voltage array has the correct number of values.
assert(numel(Vin_RMS_V) == numberOfPoints, 'Vin_RMS_V length does not match Frequency_Hz.');
```

```
% Verify that the output-voltage array has the correct number of values.
assert(numel(Vout_RMS_V) == numberOfPoints, 'Vout_RMS_V length does not match Frequency_Hz.');
```

```
% Verify that every test frequency is greater than zero.
assert(all(Frequency_Hz > 0), 'All test frequencies must be greater than zero.');
```

```
% Verify that every finite input-voltage value is greater than zero.
assert(all(Vin_RMS_V(isfinite(Vin_RMS_V)) > 0), 'Vin_RMS_V must be greater than zero.');
```

```
%% Calculate amplifier performance
```

```
% Calculate the linear voltage gain at every frequency.
VoltageGain = Vout_RMS_V ./ Vin_RMS_V;
```

```
% Convert the linear voltage gain to decibels.
Gain_dB = 20 .* log10(abs(VoltageGain));
```

```
% Calculate output power from measured RMS voltage and the resistive load.
PowerFromVoltage_W = (Vout_RMS_V.^ 2) ./ loadResistance_Ohm;
```

```
% Calculate output power from measured voltage and current when current exists.
PowerFromMeasuredCurrent_W = Vout_RMS_V .* Iout_RMS_A;
```

```
% Calculate the expected load current from output voltage and load resistance.
CalculatedLoadCurrent_A = Vout_RMS_V ./ loadResistance_Ohm;
```

```
% Calculate positive-rail droop relative to the largest measured positive rail.
PositiveRailDroop_V = max(Vrail_Pos_V, [], 'omitnan') - Vrail_Pos_V;
```

```
% Calculate negative-rail magnitude for easier comparison on a graph.
NegativeRailMagnitude_V = abs(Vrail_Neg_V);
```

```
%% Display a numerical results table
```

```
% Build a MATLAB table containing measured and calculated results.
Results = table(Frequency_Hz.', Vin_RMS_V.', Vout_RMS_V.', ...
    VoltageGain.', Gain_dB.', CalculatedLoadCurrent_A.', ...
    PowerFromVoltage_W.', Phase_deg.', ...
    'VariableNames', {'Frequency_Hz','Vin_RMS_V','Vout_RMS_V', ...
    'VoltageGain','Gain_dB','CalculatedLoadCurrent_A', ...
    'OutputPower_W','Phase_deg'});
```

```
% Display the results table in the Command Window.
disp(Results);
```

```
%% Plot voltage gain versus frequency
```

```
% -----
% TP1-TP10 INDIVIDUAL FREQUENCY-SWEEP GRAPHS
% -----
% The ten numbered test points on the annotated schematic correspond to:
% TP1 = amplifier input signal (Vin_RMS_V)
% TP2 = internal small-signal point 2 (TP2_RMS_V)
% TP3 = internal small-signal point 3 (TP3_RMS_V)
% TP4 = upper driver-path measurement point (TP4_RMS_V)
% TP5 = lower driver-path measurement point (TP5_RMS_V)
% TP6 = upper gate-drive measurement point (TP6_RMS_V)
% TP7 = lower gate-drive measurement point (TP7_RMS_V)
% TP8 = amplifier output signal (Vout_RMS_V)
% TP9 = positive supply rail (Vrail_Pos_V)
% TP10 = negative supply rail (Vrail_Neg_V)
```

```
% Store all ten measured traces in one cell array.
TP_Data = {Vin_RMS_V, TP2_RMS_V, TP3_RMS_V, TP4_RMS_V, ...
```



```
TP5_RMS_V, TP6_RMS_V, TP7_RMS_V, Vout_RMS_V, ...  
Vrail_Pos_V, Vrail_Neg_V);
```

```
% Store the corresponding plot titles.
```

```
TP_Titles = {'TP1 - Input Signal', ...  
            'TP2 - Internal Small-Signal Point', ...  
            'TP3 - Internal Small-Signal Point', ...  
            'TP4 - Upper Driver Path', ...  
            'TP5 - Lower Driver Path', ...  
            'TP6 - Upper Gate-Drive Point', ...  
            'TP7 - Lower Gate-Drive Point', ...  
            'TP8 - Amplifier Output', ...  
            'TP9 - Positive Supply Rail', ...  
            'TP10 - Negative Supply Rail'};
```

```
% Create one separate graph for each numbered test point.  
for k = 1:10
```

```
    % Read the measured trace for the current test point.  
    y = TP_Data{k};
```

```
    % Create the figure only when at least one valid measurement exists.  
    if any(isfinite(y))
```

```
        % Open a new MATLAB figure window.  
        figure('Name', TP_Titles{k});
```

```
        % Plot RMS voltage versus frequency on a logarithmic frequency axis.  
        semilogx(Frequency_Hz, y, '-o', 'LineWidth', 1.2);
```

```
        % Turn on the major and minor grid to make frequency reading easier.  
        grid on;  
        grid minor;
```

```
        % Label the horizontal axis.  
        xlabel('Frequency (Hz)');
```

```
        % Label the vertical axis.  
        ylabel('Measured RMS Voltage (V)');
```

```
        % Display the test-point name as the graph title.  
        title(TP_Titles{k});
```

```
        % Limit the displayed frequency range to the planned audio sweep.  
        xlim([100 20000]);
```

```
    end  
end
```

```
% -----  
% EXPORT CALCULATED RESULTS  
% -----
```

```
% Write the calculated results table to a CSV file for documentation.  
writetable(Results, 'amplifier_analysis_results.csv');
```

```
% Display a completion message in the MATLAB Command Window.  
disp('Analysis complete: TP1-TP10 frequency-sweep graphs generated where data are available.');
```

8. Expected Results / 8. Résultats attendus

Graph set / Ensemble de graphiques: 1) Voltage Gain vs Frequency; 2) Phase vs Frequency; 3) Output Power vs Frequency; 4) TP2-TP7 Internal RMS Voltage vs Frequency; 5) TP9/TP10 Supply Rail Magnitude vs Frequency. Frequency axis: logarithmic, 100 Hz to 20 kHz.

For the sine-wave sweep 100, 200, 500 Hz, 1, 2, 5, 10, 15 and 20 kHz, the program generates: Gain vs Frequency; Phase vs	Pour le balayage sinusoïdal 100, 200, 500 Hz, 1, 2, 5, 10, 15 et 20 kHz, le programme génère : Gain en fonction de la
--	---



Frequency when phase data exist; Output Power vs Frequency; TP2-TP7 internal RMS-voltage response; and TP9/TP10 supply-rail magnitude response. TP8 is the Vout_RMS_V output channel used for gain and power calculations.

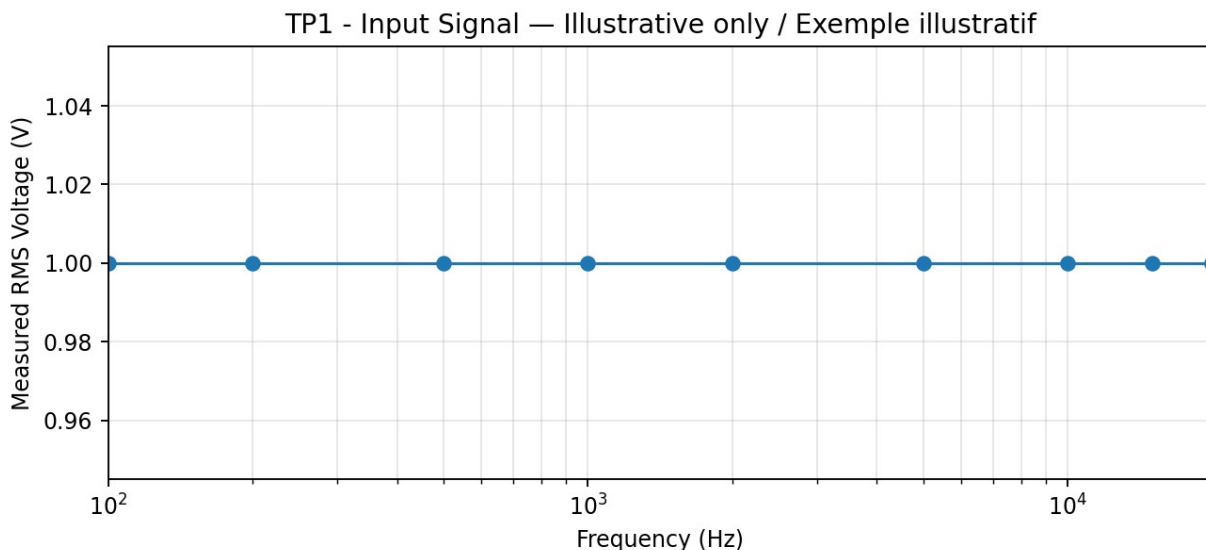
fréquence; Phase si les données existent; Puissance de sortie; réponse RMS interne TP2-TP7; et réponse des rails TP9/TP10. TP8 correspond au canal Vout_RMS_V utilisé pour les calculs de gain et de puissance.

9. TP1-TP10 Frequency-Sweep Graphs / 9. Graphiques de balayage TP1-TP10

Ten numbered schematic points are documented with ten separate graphs. The curves shown below are illustrative examples only; actual plots will be generated from bench measurements during the 100 Hz to 20 kHz sine-wave sweep.

Dix points numérotés du schéma sont documentés par dix graphiques distincts. Les courbes ci-dessous sont uniquement illustratives; les graphiques définitifs seront produits à partir des mesures au banc pendant le balayage sinusoïdal de 100 Hz à 20 kHz.

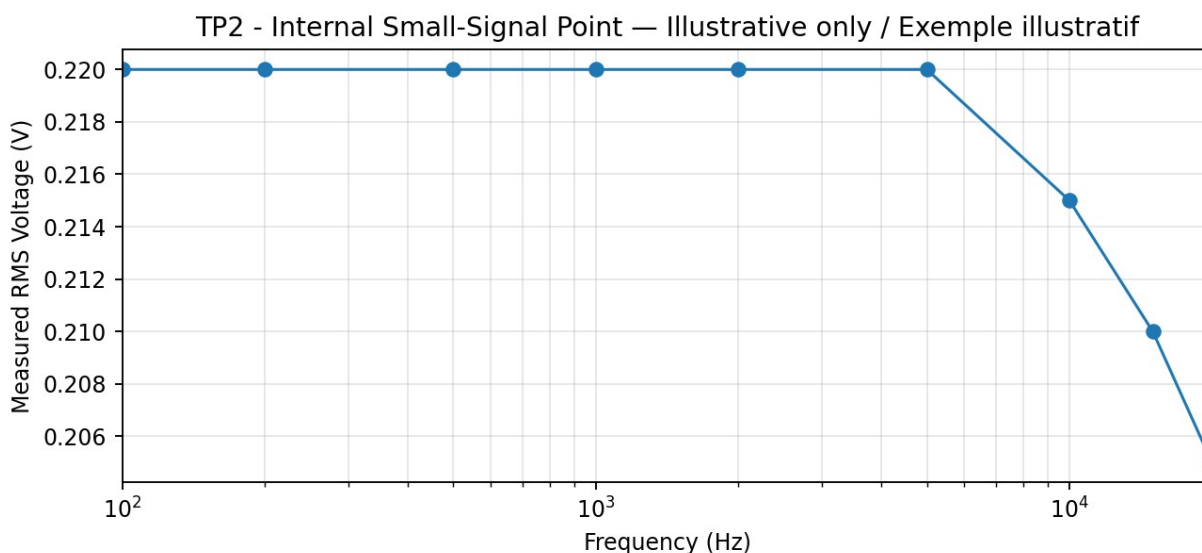
TP1 — Input Signal / Signal d'entrée



TP1 verifies that the applied sine-wave level remains constant while frequency is swept from 100 Hz to 20 kHz. A stable TP1 trace is essential because every gain calculation uses this input value as its reference. If TP1 changes with frequency, the generator or input network must be checked before interpreting amplifier response. The final graph will use the measured Vin_RMS_V data.

TP1 vérifie que le niveau sinusoïdal appliqué reste constant pendant le balayage de 100 Hz à 20 kHz. Une courbe TP1 stable est essentielle puisque tous les calculs de gain utilisent cette valeur d'entrée comme référence. Si TP1 varie avec la fréquence, le générateur ou le réseau d'entrée doit être vérifié avant d'interpréter la réponse de l'amplificateur. Le graphique final utilisera les mesures Vin_RMS_V.

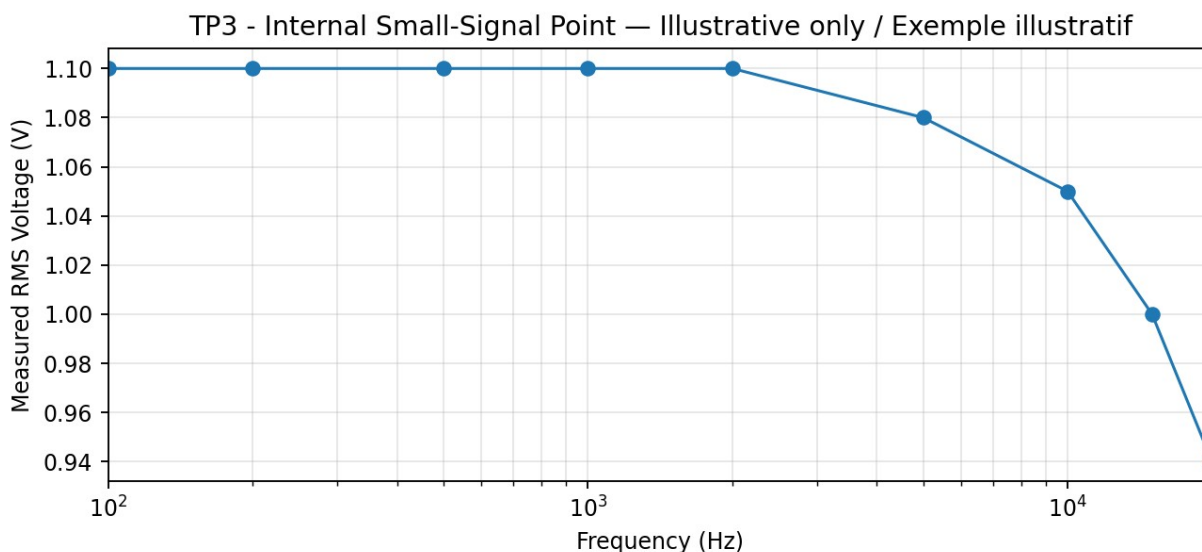
TP2 — Internal Small-Signal Point / Point interne petit signal



TP2 monitors the first internal small-signal measurement point after the input section. Its frequency response helps show whether attenuation begins before the main voltage-amplification stages. A smooth trace is expected when the front end is operating normally. Any unexpected change should be compared with TP1 and the oscilloscope waveform.

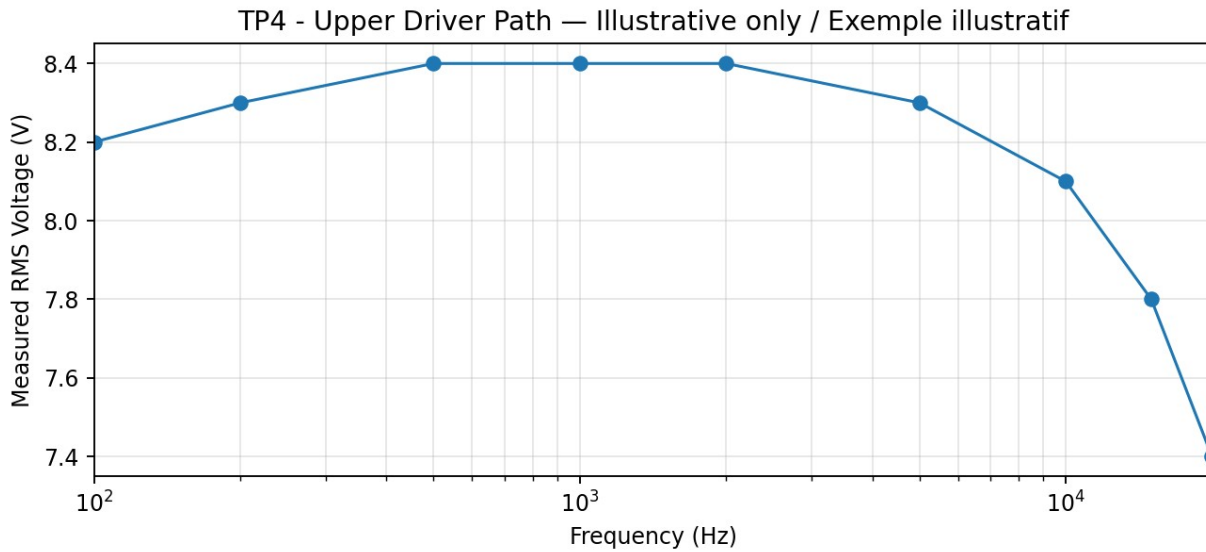
TP2 surveille le premier point interne petit signal après la section d'entrée. Sa réponse en fréquence aide à déterminer si une atténuation apparaît avant les principaux étages d'amplification en tension. Une courbe régulière est attendue lorsque l'étage d'entrée fonctionne normalement. Toute variation inattendue doit être comparée à TP1 et à la forme d'onde observée à l'oscilloscope.

TP3 — Internal Small-Signal Point / Deuxième point interne



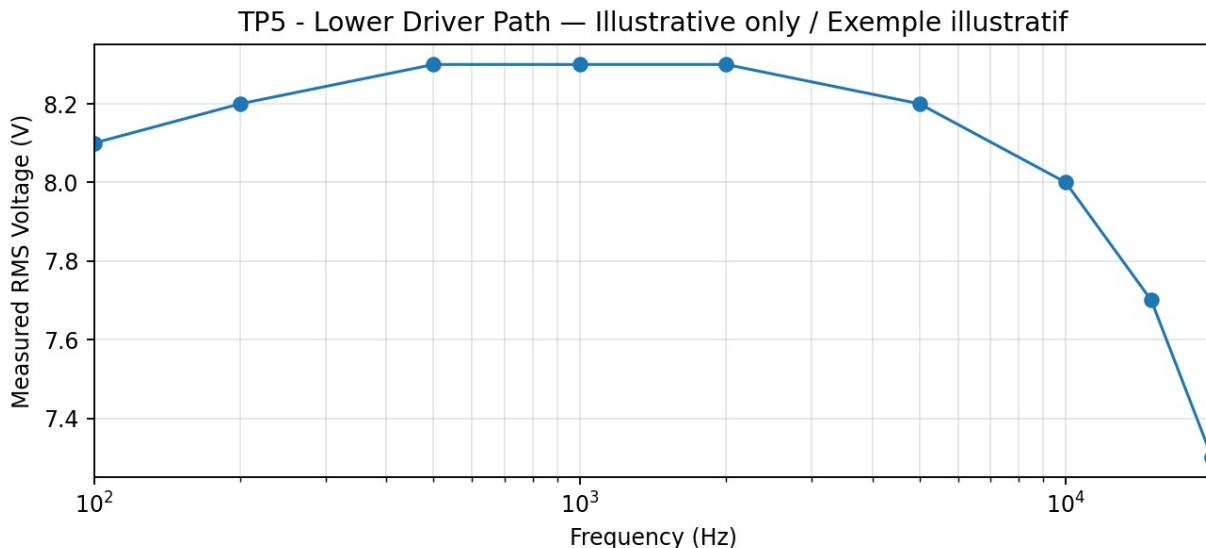
TP3 follows the signal farther into the small-signal/voltage-amplification path. Comparing TP3 with TP2 helps localize where frequency-dependent gain begins to change. The objective is not a particular voltage value but a consistent and explainable trend across the sweep. Abnormal roll-off or distortion at TP3 points toward the preceding stage or its compensation network.

TP3 suit le signal plus loin dans la chaîne petit signal/amplification en tension. La comparaison de TP3 avec TP2 permet de localiser l'endroit où le gain commence à varier avec la fréquence. L'objectif n'est pas une tension précise, mais une évolution cohérente et explicable pendant le balayage. Une atténuation ou une distorsion anormale en TP3 oriente l'analyse vers l'étage précédent ou son réseau de compensation.

**TP4 — Upper Driver Path / Voie de commande supérieure**

TP4 observes the upper driver path that feeds the positive half of the complementary output stage. Its curve should be interpreted together with TP5 to check upper/lower symmetry. A growing difference between the two paths can reveal drive imbalance or frequency-dependent loading. Oscilloscope shape remains important because RMS values alone do not reveal local oscillation.

TP4 observe la voie de commande supérieure qui alimente la moitié positive de l'étage de sortie complémentaire. Sa courbe doit être comparée à TP5 afin de vérifier la symétrie entre les deux voies. Un écart croissant peut révéler un déséquilibre de commande ou une charge dépendante de la fréquence. La forme d'onde à l'oscilloscope reste importante car une valeur RMS seule ne montre pas une oscillation locale.

TP5 — Lower Driver Path / Voie de commande inférieure

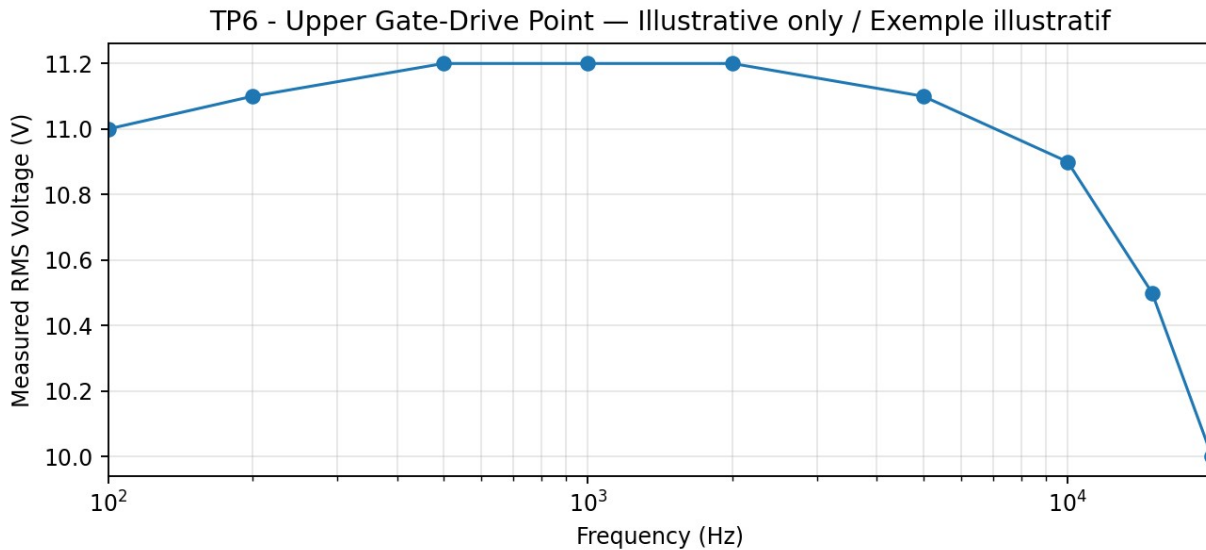
TP5 is the corresponding lower driver-path measurement. It provides the complementary comparison to TP4 over the complete 100 Hz to 20 kHz sweep. Similar trends at TP4 and TP5 support balanced driver operation, while a strong mismatch deserves investigation. The final interpretation should also consider bias and waveform symmetry.

TP5 correspond à la voie de commande inférieure. Il fournit la comparaison complémentaire à TP4 sur tout le balayage de 100 Hz à 20



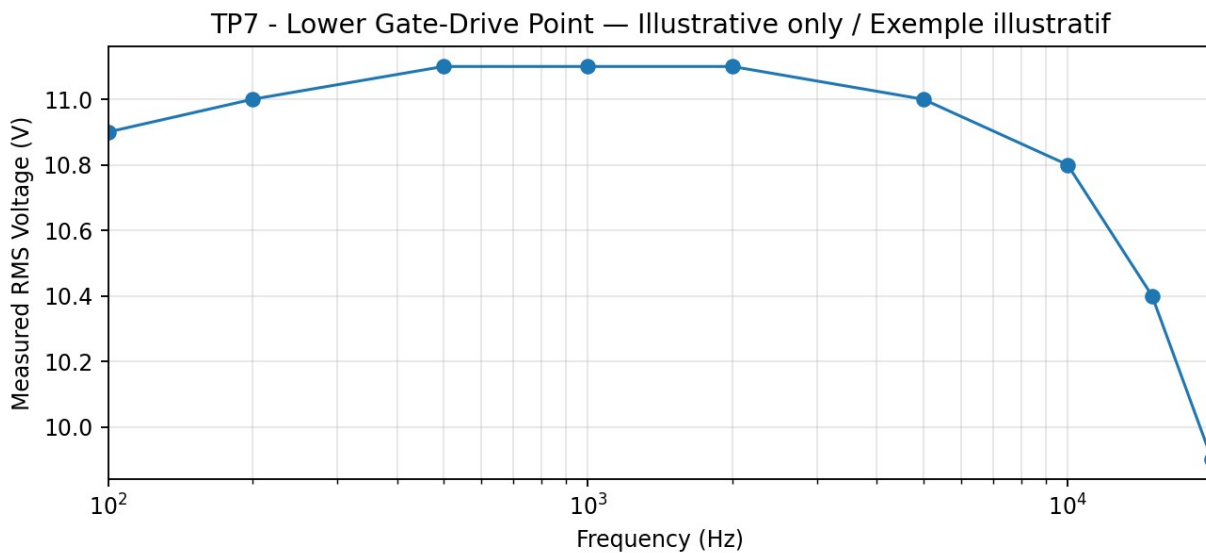
kHz. Des évolutions similaires en TP4 et TP5 indiquent un fonctionnement équilibré des drivers, tandis qu'un écart important doit être étudié. L'interprétation finale doit également tenir compte de la polarisation et de la symétrie des formes d'onde.

TP6 — Upper Gate-Drive Point / Commande de grille supérieure



TP6 monitors the upper MOSFET gate-drive region close to the parallel output bank. This point is useful for detecting loss of drive at high frequency and for checking local high-frequency activity. The RMS graph gives the frequency trend, but the same node should also be observed directly with an oscilloscope. A clean TP6 waveform is important for stable operation of the upper MOSFET bank.
TP6 surveille la commande de grille supérieure près du banc de MOSFET en parallèle. Ce point est utile pour détecter une perte de commande aux hautes fréquences et vérifier une éventuelle activité haute fréquence locale. Le graphique RMS montre la tendance avec la fréquence, mais ce nœud doit aussi être observé directement à l'oscilloscope. Une forme d'onde propre en TP6 est importante pour la stabilité du banc supérieur.

TP7 — Lower Gate-Drive Point / Commande de grille inférieure

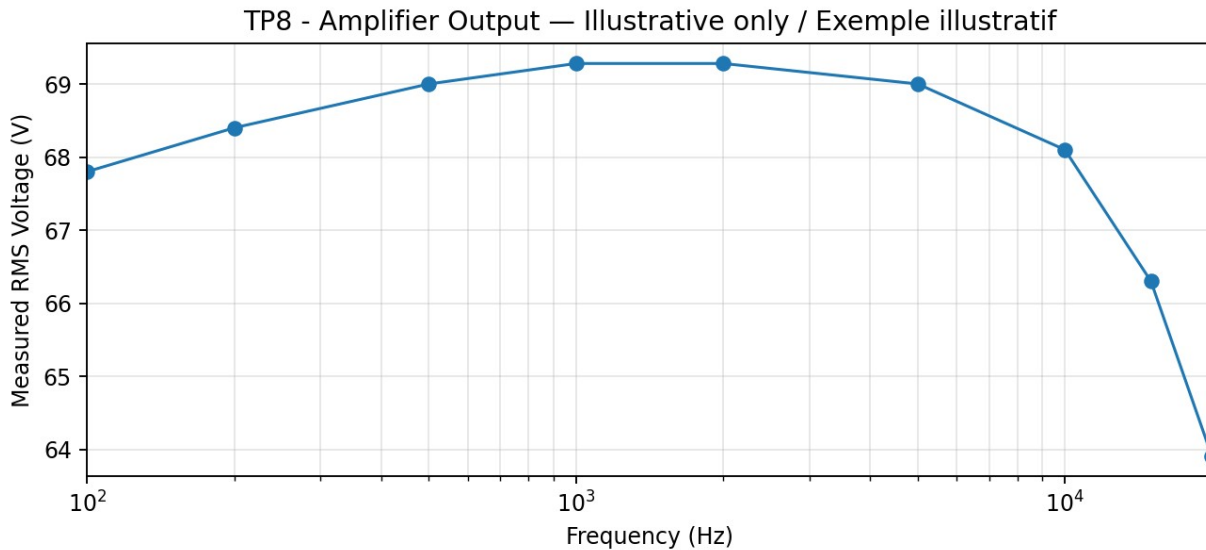




TP7 performs the same check for the lower MOSFET gate-drive path. TP6 and TP7 should be reviewed as a pair because the output stage is complementary. Differences that increase with frequency can indicate asymmetry in drive, parasitic effects or loading. Local probing is especially valuable here because gate oscillation may not be obvious at the amplifier output.

TP7 effectue la même vérification pour la commande de grille du banc inférieur. TP6 et TP7 doivent être analysés ensemble puisque l'étage de sortie est complémentaire. Des différences qui augmentent avec la fréquence peuvent indiquer une asymétrie de commande, des effets parasites ou une charge différente. Une mesure locale est particulièrement utile car une oscillation de grille peut ne pas être visible directement à la sortie.

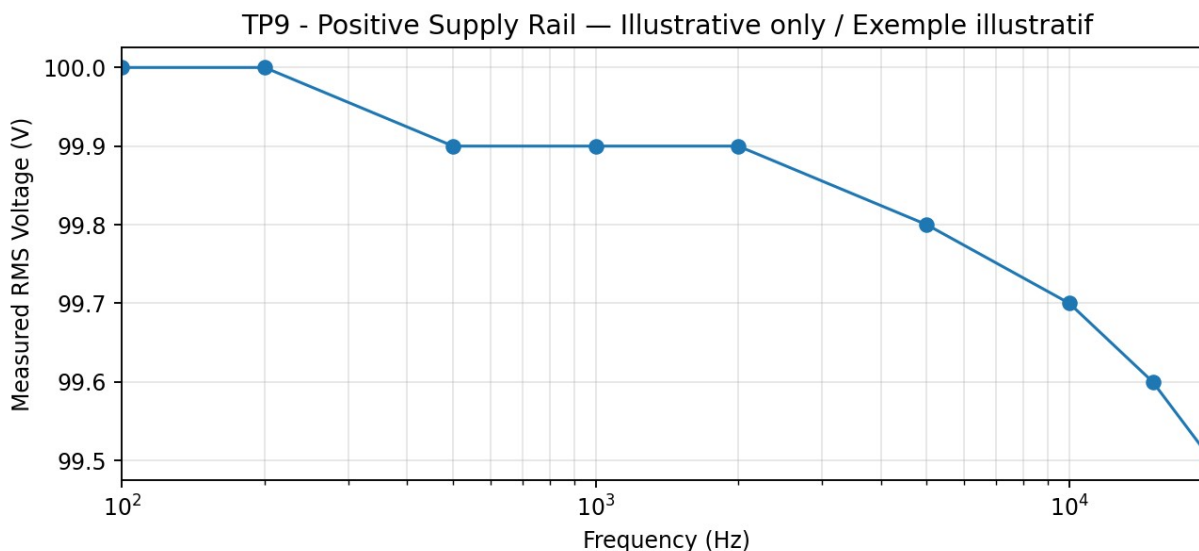
TP8 — Amplifier Output / Sortie de l'amplificateur



TP8 is the main output measurement and is used with TP1 to calculate closed-loop voltage gain. With a known resistive load, TP8 also allows output power and load current to be calculated. The curve shows whether output amplitude remains stable from 100 Hz to 20 kHz before clipping. Any reduction at the band edges must be interpreted together with the internal test points and supply rails.

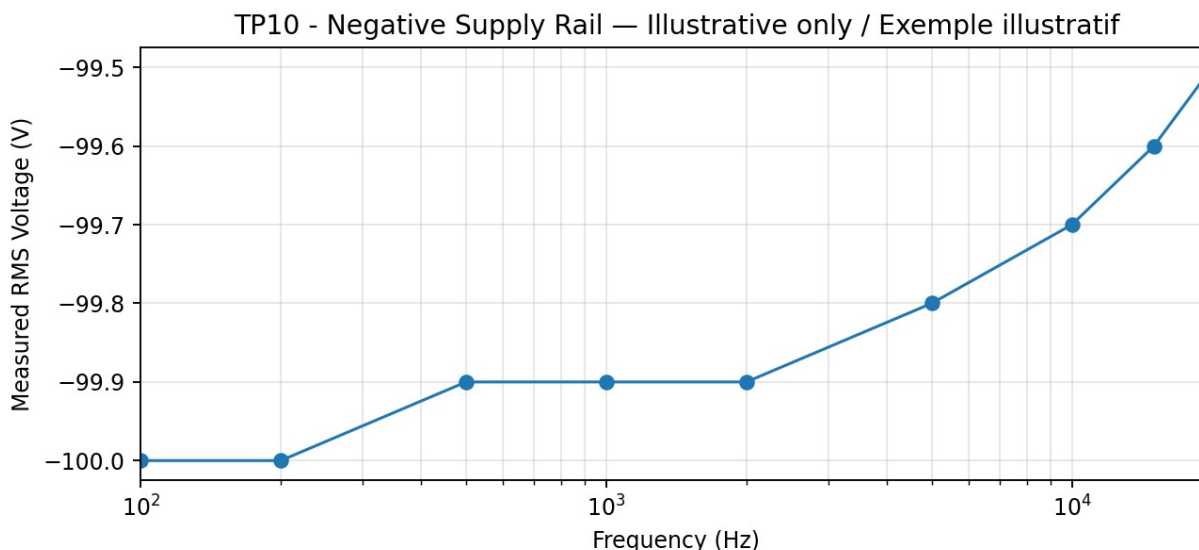
TP8 est la mesure principale de sortie et est utilisée avec TP1 pour calculer le gain en tension en boucle fermée. Avec une charge résistive connue, TP8 permet également de calculer la puissance de sortie et le courant de charge. La courbe montre si l'amplitude de sortie reste stable de 100 Hz à 20 kHz avant écrêtage. Toute diminution aux extrémités de la bande doit être analysée avec les points internes et les rails d'alimentation.

TP9 — Positive Supply Rail / Rail d'alimentation positif



TP9 records the positive supply rail while the amplifier is delivering the test signal. The rail should remain reasonably stable for a fixed output condition; excessive droop can limit positive output swing. Plotting TP9 against frequency helps distinguish amplifier bandwidth effects from power-supply limitations. The measurement must be made with suitable high-voltage instrumentation and grounding practice. TP9 enregistre le rail d'alimentation positif pendant que l'amplificateur fournit le signal d'essai. Le rail doit rester raisonnablement stable pour une condition de sortie donnée; une chute excessive peut limiter l'excursion positive. Le graphique TP9 en fonction de la fréquence aide à distinguer les effets de bande passante des limitations de l'alimentation. La mesure doit être réalisée avec un équipement adapté aux hautes tensions et une mise à la masse appropriée.

TP10 — Negative Supply Rail / Rail d'alimentation négatif



TP10 records the negative supply rail and completes the supply-symmetry check with TP9. Its magnitude should track the positive rail reasonably closely under comparable loading. Unequal droop can affect clipping symmetry and available output swing. Reviewing TP9 and TP10 separately makes supply behavior visible without hiding one rail behind the other.

TP10 enregistre le rail d'alimentation négatif et complète avec TP9 la vérification de symétrie de l'alimentation. Sa valeur absolue doit suivre raisonnablement le rail positif sous une charge comparable. Une chute différente peut influencer la symétrie de l'écrêtage et l'excursion de sortie disponible. L'analyse séparée de TP9 et TP10 permet de voir clairement le comportement de chaque rail.



10. Engineering Interpretation / 10. Interprétation technique

Compare low-, mid- and high-frequency behavior. Look for gain reduction, increasing phase shift, asymmetric driver behavior, rail droop, unexpected internal-stage changes or clipping. Do not interpret missing NaN data as a zero measurement; NaN means that the measurement has not yet been entered.

Comparer le comportement aux basses, moyennes et hautes fréquences. Rechercher une diminution du gain, une augmentation du déphasage, une asymétrie des drivers, une chute des rails, des changements inattendus dans les étages internes ou de l'écrêtage. Une valeur NaN ne représente pas zéro : elle signifie que la mesure n'a pas encore été saisie.

11. HOW TO USE THE PROGRAM / COMMENT UTILISER LE PROGRAMME

Version 1 - Manual frequency sweep / Version 1 - Balayage manuel

Edit only the Frequency_Hz array to select the desired test points, for example [100 200 500 1000 2000 5000 10000 15000 20000]. Set the sine generator manually to each frequency and record TP1-TP10 after the waveform has stabilized. Enter the measurements directly in MATLAB or import them from the CSV file. Run the script once: MATLAB then creates the ten TP frequency-response graphs from the complete sweep.

Modifier uniquement le tableau Frequency_Hz afin de choisir les fréquences d'essai, par exemple [100 200 500 1000 2000 5000 10000 15000 20000]. Régler manuellement le générateur sinusoïdal à chaque fréquence et relever TP1-TP10 après stabilisation de la forme d'onde. Saisir ensuite les mesures dans MATLAB ou les importer depuis le fichier CSV. Exécuter le programme une seule fois : MATLAB crée alors les dix graphiques TP à partir du balayage complet.

Changing the number of frequency points / Modification du nombre de fréquences

The number of graphs does not increase when more frequencies are added. Ten test points always produce ten TP graphs; adding frequencies only adds more data points to each curve. For example, a 20-frequency sweep still produces the same ten TP graphs. This makes the program suitable for a coarse first test and a denser final characterization.

Le nombre de graphiques n'augmente pas lorsque l'on ajoute des fréquences. Dix points de test produisent toujours dix graphiques TP; les fréquences supplémentaires ajoutent simplement davantage de points sur chaque courbe. Par exemple, un balayage de 20 fréquences produit toujours les mêmes dix graphiques TP. Le programme peut ainsi servir aussi bien à un premier essai rapide qu'à une caractérisation plus détaillée.

Version 2 - Automated generator sweep / Version 2 - Balayage automatisé du générateur

Version 2 connects MATLAB to a remotely controllable function generator using VISA/SCPI. MATLAB changes the sine-wave frequency automatically, waits for the selected settling time, and then asks for TP1-TP10 measurements at each step. At the end of the sweep it saves the complete results to CSV and automatically creates the ten TP graphs plus gain, output-power and optional phase graphs. The generator resource string and, if necessary, the SCPI commands must be adapted to the actual instrument.

La version 2 connecte MATLAB à un générateur de fonctions télécommandable via VISA/SCPI. MATLAB modifie automatiquement la fréquence sinusoïdale, attend le temps de stabilisation choisi, puis demande les mesures TP1-TP10 à chaque étape. À la fin du balayage, le programme sauvegarde les résultats complets en CSV et crée automatiquement les dix graphiques TP ainsi que les graphiques de gain, de puissance de sortie et, si disponible, de phase. L'adresse VISA et, si nécessaire, les commandes SCPI doivent être adaptées à l'instrument utilisé.

Full measurement automation / Automatisation complète des mesures

Automating the generator does not by itself automate TP1-TP10 acquisition. Fully automatic measurement requires suitable remotely controlled measurement hardware, such as an oscilloscope/DAQ together with enough isolated channels or an appropriate multiplexer. The supplied Version 2 therefore uses automatic generator control with manual TP entry by default and includes a clearly marked DAQ placeholder for future instrument-specific code.

L'automatisation du générateur n'automatise pas à elle seule l'acquisition TP1-TP10. Une mesure entièrement automatique nécessite un équipement de mesure télécommandable approprié, par exemple un oscilloscope/DAQ avec suffisamment de voies isolées ou un multiplexeur adapté. La version 2 fournie utilise donc par défaut le contrôle automatique du générateur avec saisie manuelle des TP et contient un emplacement clairement identifié pour une future acquisition DAQ spécifique au matériel.

12. VERSION 2 - AUTOMATED GENERATOR SWEEP / BALAYAGE AUTOMATISÉ DU GÉNÉRATEUR



The complete commented MATLAB Version 2 is reproduced below. It is intentionally instrument-neutral: the user changes the VISA resource and, where required, the generator's SCPI syntax. The frequency list remains in one clearly identified settings block, so changing the sweep does not require editing the analysis logic.

La version MATLAB 2 complète et commentée est reproduite ci-dessous. Elle reste volontairement indépendante du fabricant : l'utilisateur modifie l'adresse VISA et, si nécessaire, la syntaxe SCPI du générateur. La liste des fréquences reste regroupée dans un seul bloc de paramètres clairement identifié.

```
%% AHDS - 1 kW MOSFET Amplifier - Automated Generator Sweep
% VERSION 2 - Automated frequency generator + TP1-TP10 acquisition workflow
%
% PURPOSE
% This script automatically changes the sine-wave generator frequency through
% a user-defined list, pauses for settling, records TP1-TP10 measurements,
% saves the complete table to CSV, and creates ten frequency-response graphs.
%
% IMPORTANT HARDWARE NOTE
% MATLAB can automate the generator only if the generator supports remote
% control such as VISA/USB/LAN/GPIB/serial and SCPI (or vendor commands).
% Automatic measurement of all ten TP points additionally requires suitable
% remotely controlled measurement hardware (oscilloscope/DAQ/multiplexer).
% The default mode below automates the generator but asks the operator to
% enter TP1-TP10 measurements at each frequency. This avoids assuming a
% specific oscilloscope or DAQ that may not exist in the laboratory.
%
% SAFETY
% This amplifier uses approximately +/-100 V supply rails. Use isolated or
% differential probes where required, respect instrument input ratings, and
% never connect an oscilloscope ground clip to a non-ground power node.

%% 1. CLEAN MATLAB
clear;
close all;
clc;

%% 2. USER SETTINGS - EDIT THIS SECTION ONLY

% Select the frequencies used for the sine-wave sweep.
% Add, remove, or change values here without changing the rest of the code.
Frequency_Hz = [100 200 500 1000 2000 5000 10000 15000 20000];

% Set the sine-wave generator output amplitude.
% IMPORTANT: confirm whether your generator interprets this value as Vpp,
% Vrms, or another unit. The SCPI command below assumes Vpp.
GeneratorAmplitude_Vpp = 1.0;

% Define how long MATLAB waits after each frequency change before measuring.
SettlingTime_s = 1.0;

% Select the VISA resource string reported by MATLAB for the generator.
% Replace this example with the actual resource shown by visadevlist.
generatorResource = "USB0::0x0000::0x0000::INSTR";

% Select measurement mode.
% "manual" = generator changes automatically; operator enters TP1-TP10.
% "daq" = placeholder for a future automated oscilloscope/DAQ interface.
measurementMode = "manual";

% Define the resistive load used for output-power calculation.
loadResistance_Ohm = 8;

%% 3. CONNECT TO THE FUNCTION GENERATOR

% Display all VISA instruments visible to MATLAB.
disp(visadevlist);
```

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

```
% Open the VISA connection to the selected generator.
gen = visadev(generatorResource);

% Set a reasonable communication timeout in seconds.
gen.Timeout = 10;

% Ask the instrument for its identification string.
% Most SCPI-compatible instruments support *IDN?.
generatorID = writeread(gen, "*IDN?");

% Display the detected generator identification.
fprintf("Connected generator: %s\n", generatorID);

%% 4. CONFIGURE THE GENERATOR FOR A SINE WAVE

% Select a sine waveform.
% NOTE: SCPI syntax can vary by manufacturer; change this line if required.
writeline(gen, "FUNC SIN");

% Set the generator output amplitude.
% NOTE: this example uses Vpp.
writeline(gen, sprintf("VOLT %.6f", GeneratorAmplitude_Vpp));

% Set zero DC offset.
writeline(gen, "VOLT:OFFS 0");

% Turn the generator output on.
writeline(gen, "OUTP ON");

%% 5. PREALLOCATE TP1-TP10 MEASUREMENT ARRAYS

% Count how many frequency points will be measured.
N = numel(Frequency_Hz);

% Create an N-by-10 matrix filled with NaN.
% Each row corresponds to one frequency.
% Each column corresponds to TP1 through TP10.
TP_RMS_V = nan(N, 10);

% Create an optional phase array.
Phase_deg = nan(N, 1);

%% 6. AUTOMATED FREQUENCY SWEEP

% Step through every frequency in the user-defined frequency list.
for i = 1:N

    % Read the current test frequency.
    f = Frequency_Hz(i);

    % Send the new frequency to the generator.
    writeline(gen, sprintf("FREQ %.12g", f));

    % Display progress in the MATLAB Command Window.
    fprintf("\nFrequency %d of %d: %.12g Hz\n", i, N, f);

    % Wait for the amplifier and instruments to settle.
    pause(SettlingTime_s);

    % Check which measurement method is selected.
    if measurementMode == "manual"

        % Ask the operator to enter ten RMS voltage measurements.
```



```
% Enter values in this exact order: TP1 TP2 ... TP10.
prompt = sprintf(['Enter TP1-TP10 RMS voltages at %.12g Hz\n' ...
    'as [TP1 TP2 TP3 TP4 TP5 TP6 TP7 TP8 TP9 TP10]: '], f);

% Read the entered vector from the MATLAB Command Window.
values = input(prompt);

% Verify that exactly ten values were entered.
if numel(values) ~= 10
    error("Exactly 10 values are required for TP1 through TP10.");
end

% Store the ten values in the current measurement row.
TP_RMS_V(i, :) = values(:).';

% Ask for optional output phase relative to the input.
% Enter NaN if phase was not measured.
Phase_deg(i) = input("Enter output phase in degrees, or NaN if not measured: ");

elseif measurementMode == "daq"

    % FULL AUTOMATION PLACEHOLDER:
    % Replace the next line with instrument-specific acquisition code.
    %
    % A fully automatic system needs measurement hardware capable of
    % acquiring TP1-TP10 safely, for example:
    % - a remotely controlled oscilloscope plus switching/multiplexer,
    % - a multi-channel isolated DAQ,
    % - or another suitable automated measurement system.
    %
    % The acquisition function should return one 1x10 vector containing
    % TP1 through TP10 RMS voltages in the same order.
    error(['DAQ mode requires instrument-specific TP1-TP10 acquisition ' ...
        'code for the actual oscilloscope/DAQ/multiplexer.']);

else

    % Stop if the measurement mode name is invalid.
    error('measurementMode must be "manual" or "daq".');

end
end

%%% 7. TURN THE GENERATOR OUTPUT OFF

% Disable the generator output after the sweep is complete.
writeline(gen, "OUTP OFF");

% Release the VISA object.
clear gen;

%%% 8. ASSIGN MEASUREMENT COLUMNS TO NAMED TP VARIABLES

% Copy TP1 measurements.
TP1_RMS_V = TP_RMS_V(:,1);

% Copy TP2 measurements.
TP2_RMS_V = TP_RMS_V(:,2);

% Copy TP3 measurements.
TP3_RMS_V = TP_RMS_V(:,3);

% Copy TP4 measurements.
TP4_RMS_V = TP_RMS_V(:,4);
```

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

```
% Copy TP5 measurements.
TP5_RMS_V = TP_RMS_V(:,5);

% Copy TP6 measurements.
TP6_RMS_V = TP_RMS_V(:,6);

% Copy TP7 measurements.
TP7_RMS_V = TP_RMS_V(:,7);

% Copy TP8 measurements.
TP8_RMS_V = TP_RMS_V(:,8);

% Copy TP9 measurements.
TP9_RMS_V = TP_RMS_V(:,9);

% Copy TP10 measurements.
TP10_RMS_V = TP_RMS_V(:,10);

%% 9. CALCULATE MAIN AMPLIFIER RESULTS

% Calculate closed-loop voltage gain using TP8 output divided by TP1 input.
VoltageGain = TP8_RMS_V ./ TP1_RMS_V;

% Convert voltage gain to decibels.
Gain_dB = 20 .* log10(abs(VoltageGain));

% Calculate output power into the selected resistive load.
OutputPower_W = (TP8_RMS_V.^ 2) ./ loadResistance_Ohm;

% Calculate load current from output voltage and load resistance.
CalculatedLoadCurrent_A = TP8_RMS_V ./ loadResistance_Ohm;

%% 10. BUILD AND SAVE THE COMPLETE MEASUREMENT TABLE

% Build a MATLAB table containing frequency, TP1-TP10, phase, and results.
Results = table(Frequency_Hz(:, ...
    TP1_RMS_V, TP2_RMS_V, TP3_RMS_V, TP4_RMS_V, TP5_RMS_V, ...
    TP6_RMS_V, TP7_RMS_V, TP8_RMS_V, TP9_RMS_V, TP10_RMS_V, ...
    Phase_deg, VoltageGain, Gain_dB, OutputPower_W, CalculatedLoadCurrent_A, ...
    'VariableNames', {'Frequency_Hz', ...
    'TP1_RMS_V','TP2_RMS_V','TP3_RMS_V','TP4_RMS_V','TP5_RMS_V', ...
    'TP6_RMS_V','TP7_RMS_V','TP8_RMS_V','TP9_RMS_V','TP10_RMS_V', ...
    'Phase_deg','VoltageGain','Gain_dB','OutputPower_W', ...
    'CalculatedLoadCurrent_A'});

% Display the completed table.
disp(Results);

% Save the measurement table to a CSV file.
writetable(Results, "AHDS_automated_frequency_sweep_results.csv");

%% 11. CREATE TEN TP1-TP10 FREQUENCY-RESPONSE GRAPHS

% Store all ten traces in a cell array for convenient plotting.
TP_Data = {TP1_RMS_V, TP2_RMS_V, TP3_RMS_V, TP4_RMS_V, TP5_RMS_V, ...
    TP6_RMS_V, TP7_RMS_V, TP8_RMS_V, TP9_RMS_V, TP10_RMS_V};

% Store the graph titles.
TP_Titles = {'TP1 - Input Signal', ...
    'TP2 - Internal Small-Signal Point', ...
    'TP3 - Internal Small-Signal Point', ...
    'TP4 - Upper Driver Path', ...
    'TP5 - Lower Driver Path', ...
```



```
'TP6 - Upper Gate-Drive Point', ...  
'TP7 - Lower Gate-Drive Point', ...  
'TP8 - Amplifier Output', ...  
'TP9 - Positive Supply Rail', ...  
'TP10 - Negative Supply Rail');
```

```
% Generate exactly one graph for each test point.  
for k = 1:10
```

```
    % Open a new figure.  
    figure('Name', TP_Titles{k});
```

```
    % Plot all measured frequency points on one logarithmic frequency axis.  
    semilogx(Frequency_Hz, TP_Data{k}, '-o', 'LineWidth', 1.2);
```

```
    % Enable major and minor grid lines.  
    grid on;  
    grid minor;
```

```
    % Label the horizontal axis.  
    xlabel('Frequency (Hz)');
```

```
    % Label the vertical axis.  
    ylabel('Measured RMS Voltage (V)');
```

```
    % Add the test-point title.  
    title(TP_Titles{k});
```

```
    % Use the actual first and last frequencies from the user settings.  
    xlim([min(Frequency_Hz) max(Frequency_Hz)]);
```

```
end
```

```
%% 12. CREATE DERIVED PERFORMANCE GRAPHS
```

```
% Create the closed-loop gain graph.  
figure('Name', 'Closed-Loop Gain');  
semilogx(Frequency_Hz, Gain_dB, '-o', 'LineWidth', 1.2);  
grid on;  
grid minor;  
xlabel('Frequency (Hz)');  
ylabel('Gain (dB)');  
title('Closed-Loop Voltage Gain - TP8 / TP1');
```

```
% Create the output-power graph.  
figure('Name', 'Output Power');  
semilogx(Frequency_Hz, OutputPower_W, '-o', 'LineWidth', 1.2);  
grid on;  
grid minor;  
xlabel('Frequency (Hz)');  
ylabel('Output Power (W)');  
title('Calculated Output Power vs Frequency');
```

```
% Create the phase graph only when phase measurements exist.  
if any(isfinite(Phase_deg))  
    figure('Name', 'Output Phase');  
    semilogx(Frequency_Hz, Phase_deg, '-o', 'LineWidth', 1.2);  
    grid on;  
    grid minor;  
    xlabel('Frequency (Hz)');  
    ylabel('Phase (degrees)');  
    title('Output Phase vs Frequency');  
end
```



AHDS

Annissoff Hardware Development System
Design once. Reuse forever.

Email: ahds@engineer.com

%% 13. FINISH

% Confirm that the complete automated sweep has finished.

disp("Automated generator sweep complete.");

% Remind the user where the results were saved.

disp("Results saved to AHDS_automated_frequency_sweep_results.csv.");